VEXLEARN

TEMPLATE FOR LEARNING

VEXLearn Guide

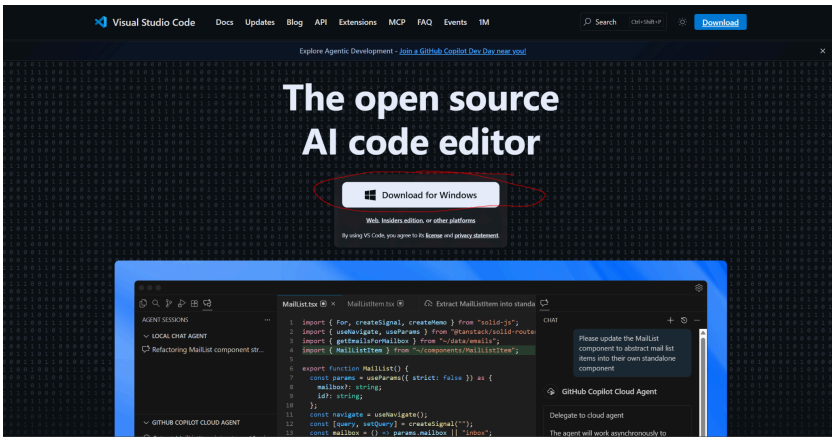
Table of Contents (Click to Go to Section)

Table of Contents (Click to Go to Section) **2**
Setup Guide **3**
Installing Code Editing Software (Visual Studio Code Example) 3
Installing the VEX VS Code Extension 4
Installing the Template 6
For Learning Software Only 7
For Use 7
Tips for Coders **9**
Writing Autonomous Functions 9
Rely on a Template 10
Your Role not being Valued 10
Engineering Notebook & Interviews 11
C++ Fundamentals **12**
Datatypes and Purpose 12
Basic Syntax 12
General Syntax 12
Initializing Variables and Assigning Values 13
Mathematical, Assignment, Comparisons, and Logical Operators 13
While Loops, For Loops, and If-Statements 15
Enumerators (enum) 16
Program Structure 17
Header Files 17
File Integration Through #include 18
Creating Functions and Calling Functions 18
Namespaces 19
VEX Concepts **21**
Object Oriented Programming and VEX Objects 21
Object Oriented Programming 21
VEX Objects 22
How Ports Work 23
Gear Ratio’s Effect on Software 24
6 vs 12 Watt Motors 24
VEX Threads 25
Proportional–Integral–Derivative Controller 25
Integral Windup 26
Volt Control vs RPM Control 27
Absolute Heading & Changing Inertial Reading 27
Inertial.setHeading(); 28

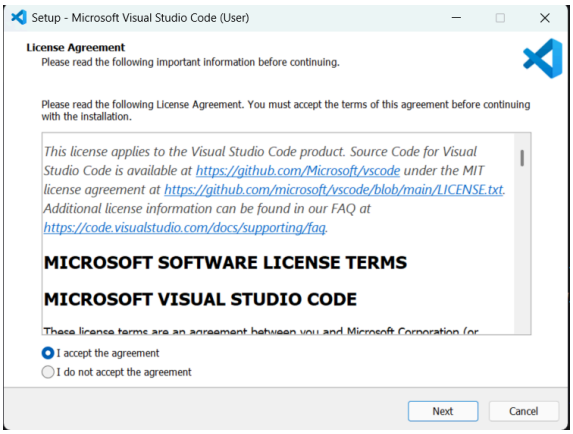
Setup Guide

Installing Code Editing Software (Visual Studio Code Example)

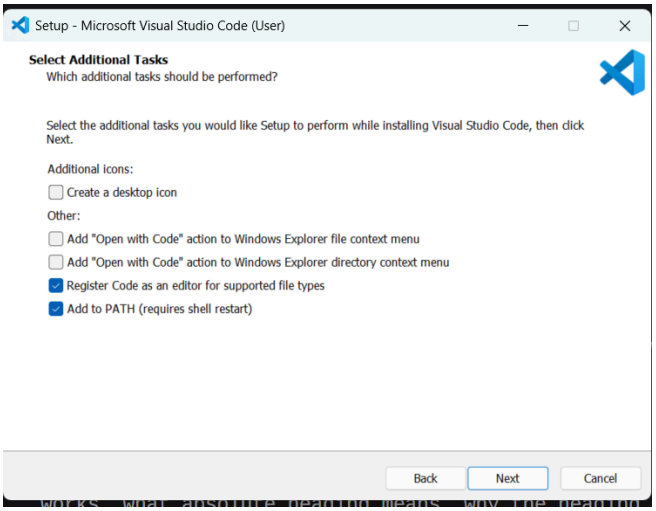
1. Open your browser (Chrome, edge, firefox, etc.)
2. In the address bar, type “code.visualstudio.com”, then enter
3. Click the “Download for Windows” button



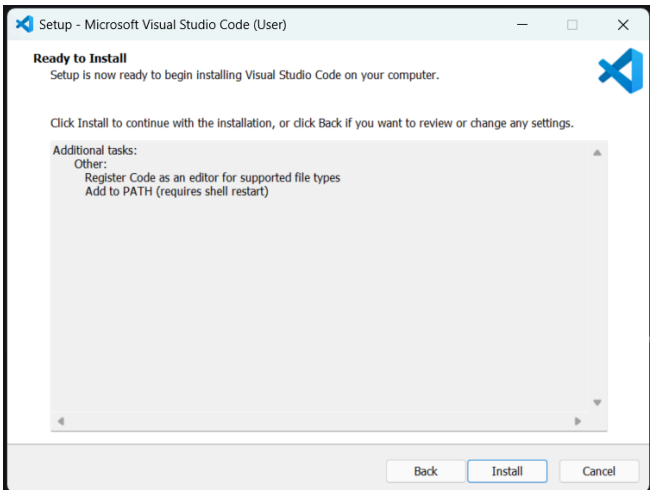
4. Wait for the file to download
5. Open the downloaded file
6. Accept the agreement



7. Choose add to PATH. Whatever else you select in this page does not matter

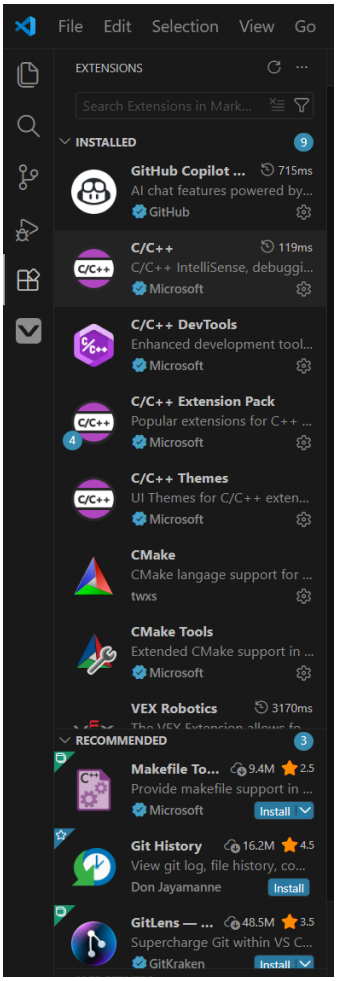


8. Click install

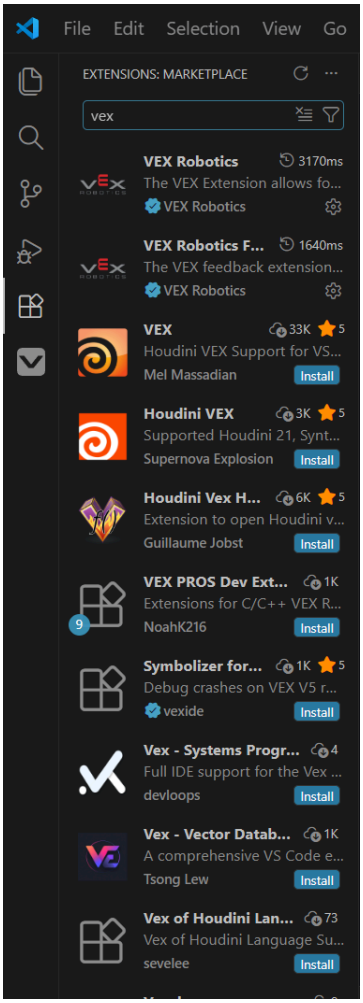


Installing the VEX VS Code Extension

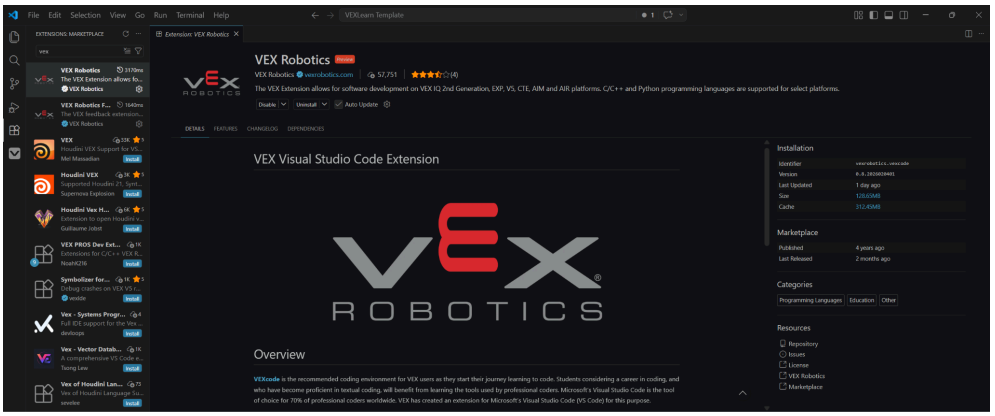
1. Open your code editing software
2. Go to the extensions tab


File
Edit
Selection
View
Go
EXTENSIONS
Search Extensions in Mark...
INSTALLED
9
GitHub Copilot ...
715ms
AI chat features powered by...
GitHub
C/C++
119ms
C/C++ IntelliSense, debuggi...
Microsoft
C/C++ DevTools
Enhanced development tool...
Microsoft
C/C++ Extension Pack
Popular extensions for C++ ...
Microsoft
C/C++ Themes
UI Themes for C/C++ exten...
Microsoft
CMake
CMake langage support for ...
twxs
CMake Tools
Extended CMake support in ...
Microsoft
VEX Robotics
3170ms
RECOMMENDED
3
Makefile To...
9.4M
2.5
Provide makefile support in ...
Microsoft
Install
Git History
16.2M
4.5
View git log, file history, co...
Don Jayamanne
Install
GitLens — ...
48.5M
3.5
Supercharge Git within VS C...
GitKraken
Install

3. Search “vex”



4. Install “VEX Robotics”



5. Wait for the extension to install

Installing the Template

1. Prerequisites
    a. You have code editing software (Visual Studio Code, Antigravity, etc.)
    b. You have the VEX extension installed on the code editing software
2. Download the ZIP file.
3. Extract the ZIP file once you have downloaded it.

4. Open your code editing software, click on the file on the top right, open the folder, then open the extracted folder from the ZIP file.
5. Let your code editing software install the dependencies it needs
    a. If the “Do you want to install Microsoft’s Makefile tools?” pops up, DO NOT install it, it will break the template

For Learning Software Only

If you just want to see how the code works and looks like, you do not need any extra setup beyond installing the template.

For Use

If you want to use the template on a robot, your robot has to fit the following prerequisites:

a. Chassis has to be tank-drive design
    i. No x-drive
    ii. No h-drive
    iii. No mecanum
b. All wheels for chassis must be the same size
c. Chassis has to have a clear split between motors for left chassis and motors for right chassis
d. Omni wheels and traction wheels are both fine
e. Robot has to have an inertial sensor (most robots will have this)
f. Only uses V5 motors
g. Only uses one controller

Additionally, your software also must have the following prerequisites:

a. Uses VEX VS Code extension
    i. PROS is not supported
b. Only used for V5 platform
    i. IQ or EXP is not supported

If you fit all the prerequisites, follow these steps to setup the template (if don’t know about something in the setup, go to table of contents and look up the guides):

1. Go to vex.h and change wheel_2r to the diameter of your wheels
2. Go to robotConfig.cpp
    a. Add or remove motors (you can delete all the example motors)
        i. Make sure you have the correct gear ratio
        ii. Make sure you have the correct port number
        iii. Make sure you have the correct direction
    b. Add or remove sensors (you can delete all the example sensors)
        i. Make sure you have the correct port number
    c. Add or remove pistons (you can delete all the example pistons)
        i. Make sure you have the correct three wire port
    d. Add or remove motors from the motor groups

        i. Make sure your motor names match
    e. Basically make sure the robotConfig matches your robot
3. Go to robotConfig.h
    a. Initialize the same objects you created in robotConfig.cpp, follow the structure explanation for help
4. Go to movement.cpp
    a. The gear ratio in the move function is set to a default 4 / 3, check with your builder on what the gear ratio is then replace the 4 / 3 with your gear ratio
5. Go to drive.cpp
    a. You can delete all the examples (don't delete the chassis control)
    b. Write the button mappings
6. Go to main.cpp
    a. Change the pre_auton to calibrate your sensors
7. Go to display.cpp
    a. Remove motors from the motors screenState function that don't exist
    b. Go to the teams screenState function and change the placeholders (optional)
    c. Change the software version number if you want to track versions (optional)
8. Compile the project to make sure no errors, warnings are fine
9. Run the project on the robot and make sure it works
10. Go to drive.cpp (this part is recommended to be done with your driver)
    a. Change the button mappings to your driver's preference
    b. Change the deadband to your driver's preference
    c. Change the sensitivity to your driver's preference
11. Go to auto.cpp
    a. Write a simple PID tuning function like

```
void test() {
        move(100, 0, 8);
}
```

12. Go to auto.h
    a. Add your PID tuning function, follow the structure explanation for help
13. Go to movement.cpp
    a. Start tuning PID for move() and turn()
    b. Reference the PDF's PID section for help
14. When you finish this setup, your robot is ready. You can start writing the autonomous functions.

Tips for Coders

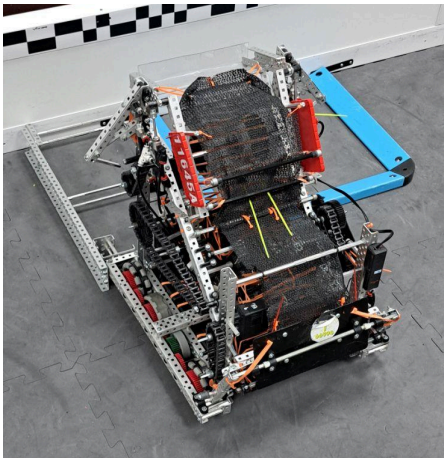
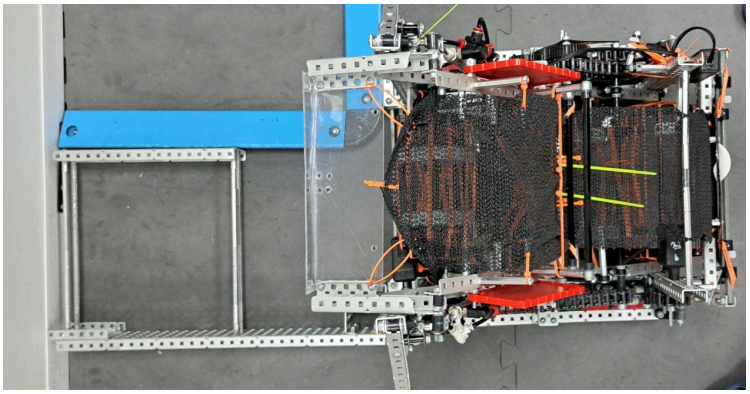
Over the course of my two year journey in VEX, I have dealt with many situations like the mentor not being helpful, the mentor blaming the coder for limitations caused by hardware, and burnout. These experiences were not very fun, and I want to share my experience with any coder who might be in a similar situation.

Writing Autonomous Functions

First, I want to give some tips on the most time consuming part of being the coder for the team, which is writing the autonomous function. An autonomous function is used during competitions 15 second autonomous or skills autonomous. Autonomous code makes your robot follow a set path you wrote in the function. If you have ever used scratch, you will know that there are change x by and change y by functions. When you call the movement functions inside the autonomous functions, it does the same thing. When writing VEX autonomous functions, you should follow these rules:

1. Writing autonomous functions is a very tiring, very boring, and very time intensive task. I personally spent weeks just to perfect the 15 second autonomous functions. You get about 90% of the way there in the first few hours you spend on tuning the autonomous function, but to increase reliability and performance it often takes a lot more time.
2. Make sure that your PID movement functions are PROPERLY tuned. Ideally, the margin of error for traveling one meter is smaller than 0.5 centimeter and the margin of error when traveling three meters is smaller than three centimeters. You DO NOT, WILL NOT, and WILL NEVER save yourself any time by having poorly tuned PID movement functions.
3. If your robot's battery is lower than 75%, it WILL cause deviations in movement. As such, you should NEVER tune your autonomous functions once the battery is lower than 75%. Trust me, you don't save yourself any time by avoiding changing the battery.
4. If your robot uses pistons and has gas tubes, make sure to refill them at least once every two runs. I am speaking from personal experience, you most likely do not want your route to go smoothly just to fail because you didn't refill the gas tubes. You also don't save yourself any time.
5. If you are the ONLY programmer in the team, it is important that you NEVER solo the autonomous writing process. This is because when you have to get up from your chair, put everything on the field back to its position, then get back and change something, it gets extremely tiresome very quickly. Make sure you have a buddy to help you do those things, not only does it make you write the autonomous functions faster, you also prevent burnout by having someone to talk to and not having to put the robot and stuff back. If you are part of a larger programming group then make sure that someone is in charge of cleaning the field for you.
6. If you have been stuck on a part for some time, use your phone to record the run. This will SAVE you a TON of time because you can replay a part multiple times and find out what the problem is much quicker. If you have a buddy, tell him/her to record for you. If you don't, grab a recording stand and film it yourself.

7. Make sure you have at least a week to write autonomous functions without interruption from your builder or your driver when you are tuning the functions. I know some programmers that are wizards and could whip up a perfect autonomous function in a single afternoon. I also know a programmer that takes two months to write an autonomous function. You should never be discouraged by how long it takes for you to write the autonomous program, especially if you are doing it solo (your teammates aren't helping you clean the field). Your task order should be tuning PID functions --> designing the teamwork autonomous routes (if you don't know how you can watch what other teams are doing) --> write the teamwork autonomous (15 second one) based on the routes you designed --> designing the skills autonomous routes (IF YOU HAVE TIME) --> write the skills autonomous routes (IF YOU HAVE TIME). I personally would argue that teamwork autonomous is more important as your skills autonomous really has to be insanely good for you to win any award.
8. An aligner (physical aligning device to make sure your robot placement is nearly identical every run) is VERY IMPORTANT. Tell your builder to make one for you or make one yourself.

*Examples of aligner and robot from my Push Back year

If you are planning on writing code skills, you should use the JAR Template or the LemLib template. These templates have functions for odometry written and ready for you to use. Odometry is very important in skills because you will be covering much longer distances than in the 15 second autonomous. My advice, switch to JAR Template or LemLib for skills as my template is mostly focused in teaching and simpler tasks.

Rely on a Template

If you don’t have the time to write your own custom VEX software, don’t be afraid to rely on a template like JAR Template and LemLib for competitions. My mentor encouraged our team (which was a small three person team) to write our own software. If your mentor tells you this and your team is very small, ignore him and just rely on a template. Writing your own code requires a lot of testing, debugging, and time investment. This is simply not worth the time investment for a small team when you also need time to write the autonomous functions.

I was not informed about the existence of templates until very late in my second year. My advice, rely on a template if you don’t have the time, but the performance ceiling of a custom software is much higher when written well compared to any template.

Like I said in the autonomous functions section, the best templates out there are JAR Template and LemLib. Both templates are competition focused templates that only require you to set up the template and tune the PID. Those templates can also handle odometry and coordinate systems and are very reliable.

Your Role not being Valued

As I said, I was part of a very small VEX team. We only had three people, one person building the robot (the builder), one person writing the software and autonomous functions (me), and one person only in charge of driving. In our team's first year, we struggled because we worked together poorly. The robot took months to build and only finished days before the competition. Admittedly, I could have helped in building the robot, but whatever I built, the quality wasn't high enough to be put on the robot and I sucked at building. There were many competitions where I had to stay up very late the night before the competition (3~4 AM) to write the autonomous functions alone because there simply wasn't enough time. I was also soloing the software for the first year of the competition (no buddy to help set up the field or record), which increased burnout and discouraged me from writing software.

When our team inevitably lost in competitions, I was blamed because the autonomous programs, which were very important for High Stakes and Push Back, weren't very strong or reliable. If you follow my first two section's tips, you should be able to comfortably avoid these problems. If you can't follow the first two tips due to team size limitations, then I would say just don't stress over VEX. The time investment you are going to have to put in because you are the only person writing the program and the autonomous programs just to win a few awards is just not worth it.

Engineering Notebook & Interviews

If your role is only limited to programming, then you won't have a lot to write on the engineering notebook. For me, what I wrote was the evolution of my software, how each software worked, why I wrote them that way, and the timeline of the evolution of my software. Additionally, you can include pictures of the results of your autonomous program's final position (which is a great way to increase page count). I was able to write about 150 pages for the engineering notebook for the Push Back season.

For interviews, I'm going to be very upfront with you, this template will not help your interviews. First and foremost, the judges have heard about PID control and PID based movements hundreds of thousands of times already, if that is only what you tell them, they will not be interested. The display gimmick is nice, but ultimately, anybody can write it easily and it isn't very impressive. The judges want to hear something interesting, something unique to you. This is up to you to decide and make, and I can not help you with it.

C++ Fundamentals

C++ fundamentals teaches the basic syntax of C++, what variables and data types mean, the loops used in the template, boolean logic (true / false), what #include means, header files and what they do, namespaces, and more advanced topics like enum. I won't cover everything since some things are almost never used in VEX, such as printing strings using iostream. These are fundamental to using the template and writing VEX software in general, so it is important that if you do not know, you learn them quickly.

Datatypes and Purpose

When we say datatypes, we mean the type of data something can hold. For example, 8 is an integer datatype, 8.8 is a double (or float but we generally don't use that) datatype, "Hello!" is a string datatype, and false is a boolean datatype. The datatypes that are used in the template are as follows:

Datatype	Purpose	Examples
int	Holds an integer value	1, 2, -5, 98729
double	Holds a numerical value that can have decimals	2.2, 5.7, 1.0, 98794.48
bool	Holds a true or false value	true, false
string	Holds a text	"Hello!", "Coding is fun"
void	Used more in function return types, holds no value	None
enum	A custom datatype	MENU, Hello123, John

*These aren't all the datatypes that exist, but the ones that are mainly used or used in the template

Datatypes are commonly used in function return value types and declaring variables.

Basic Syntax

Basic syntax teaches you how to declare variables, mathematical operators, and loops as well as the syntax and general structure of the program.

General Syntax

Almost everything you write has to end with a semicolon. Just like how a period ends a sentence, a semicolon ends a sentence in C++. The exceptions are for loops, while loops, and if-statements, which don't require a semicolon. You can see that all my variable initialization examples end in a semicolon and all the loops and if-statements do not.

Additionally, the compiler compiles starting from line 1 until the end. If you call a variable or a function before it exists, then your program will run into an error. Sometimes you might not

run into an error because you initialized the variable in the header file also. While this may work, it is still good practice to make sure you have initialized something before calling it.

Initializing Variables and Assigning Values

Variables are always initialized through the format:

```
data_type variable_name;
```

For example you can initialize the following variables:

```
int age;
bool sleeping;
double average;
string greeting;
```

Generally you declare a variable to use it, so that means you will have to assign a value to it. To assign a value to a variable, it would look like this:

```
data_type variable_name = value;
```

To assign values when initializing a variable, it would be:

```
int age = 25;
bool sleeping = true;
double average = 8.8;
string greeting = "Hello!";
```

To assign values to a variable that has already been initialized, it would be:

```
age = 25;
sleeping = true;
average = 8.8;
greeting = "Hello!";
```

Mathematical, Assignment, Comparisons, and Logical Operators

Mathematical operators are the syntax that computes a mathematical equation. Assignment is syntax used for assigning values. Comparisons are for comparing two values.

Mathematical Operators

Operator / Syntax	Purpose	Examples
+	Adds two values	5 + 2, speed1 + speed2
-	Subtracts two values	5 - 2, speed1 - speed2
/	Divides two values	5 / 2, speed1 / speed2

*	Multiplies two values	5 * 2, speed1 * speed2
%	Gets the remainder of two values after division (only works for int datatype!)	5 % 2 (returns 1), speed1 % speed2 (returns remainder)
++	Increases the value by 1	5++ (= 6), speed1++ (= speed1 + 1)
--	Decreases the value by 1	5-- (= 4), speed1-- (= speed1 - 1)

Assignment Operators

Operator / Syntax	Purpose	Examples
=	Assigns a value	int dogs = 3, double speed1 = 4
+=	Adds a value, then assigns the result	x += 3 (= x + 3), speed1 += 6 (= speed1 + 6)
-=	Subtracts a value, then assigns the result	x-= 3 (= x - 3), speed1 -= 6 (= speed1 - 6)
/=	Divides a value, then assigns the result	x /= 3 (= x / 3), speed1 /= 6 (= speed1 / 6)
*=	Multiplies a value, then assigns the result	x *= 3 (= x * 3), speed1 *= 6 (= speed1 * 6)

Comparison Operators

Operator / Syntax	Purpose	Examples
==	Checks if two values are equivalent	5 == 3 (false), 5 == 5 (true)
!=	Checks if two values aren't equivalent	5 != 3 (true), 5 != 5 (false)
>	Checks if left value is greater than right value	5 > 3 (true), 3 > 5 (false)
<	Checks if right value is greater than left value	5 < 3 (false), 3 < 5 (true)
>=	Checks if left value is greater than or equal to right value	5 >= 3 (true), 3 >= 5 (false), 5 >= 5 (true)
<=	Checks if left value is less than or equal to right value	5 <= 3 (false), 3 <= 5 (true), 3 <= 3 (true)

Logical Operators

Operator / Syntax	Purpose	Examples
&&	Returns true if both left and right values are true	false && true (false), true && true (true)
||	Returns true if either left or right values are true	false || true (true), false || false (false)
!	Inverts the value	!false (= true) !true (= false)

While Loops, For Loops, and If-Statements

These are used to do something repeatedly or do something only under some circumstance.

While Loops

While loops typically look like this:

```
while(statement) {
        //something
}
```

Since the while loop's comparison already compares to true, sometimes you will see:

```
while(true) {
        //something
}
```

If we expand it, it simply becomes this:

```
while(true == true) {
        //something
}
```

Your statement can be anything:

```
while(dogs >= 3) {
        //something
}

while(pistonsAreActivated) {
        //something
}

while(pistonsAreActivated == true) {//something}
```

For Loops

For loops follow this syntax:

```
for(int i = 0; i < value; i++) {
	//something
}
```

This basically means if the variable i, which is initialised as 0, is smaller than the value, keep running and increase i by 1 every time the loop runs once.

The value can be anything, the condition can be anything, and the index variable (i) can also be anything:

```
for(int speed = 10; speed > 0; speed--) {
	//something
}

for(int speed = 10; speed > 0; speed--) {//something}
```

If-Statements
If-statements follow this syntax:

```
if(condition) {
	//something
}
```

The something only executes if the condition is true, else everything in the if-statement is not run. Here are a few examples:

```
if(pistonsAreActivated == false) {
	pistonsAreActivated = true;
}

if(speedNow > 6) {speedNow = 6;}
```

Ternary Operators
Ternary operators are basically if-statements but more concise. It follows this syntax:

```
variable = (condition) ? expressionTrue : expressionFalse;
```

This basically means evaluate the condition, if the condition is true, run expressionTrue, else run expressionFalse. You can always avoid using this by using if-statements, which are more beginner friendly.

Enumerators (enum)

Enum is a custom data type. You can avoid using this unless you really need to. An example is in display.cpp, where the screenstates are enum datatypes. The program could be written

so the screenstates are string datatypes instead. The enum is just a demonstration. The structure should always be:

```
enum data_type_name {
  data_option,
  data_option,
  data_option
};
```

For example, using the display.cpp screenstate enum:

```
enum ScreenState {MENU, MOTORS, BATTERY, TEAM, SOFTWARE};
```

Then you can use the enum datatype like how you would use a normal datatype. For example, initializing a variable using the custom enum datatype would look like:

```
ScreenState currentScreen = MENU;
```

You can always avoid this, so don't force yourself to use it if there are easier alternatives.

Program Structure

A program can be split into many different files that work together. In the template, you mainly see two divisions, one is the .cpp files, and the other is the .h header files. In this section, you will learn about header files, integrating multiple files through #include, abstraction through functions, and namespaces.

Header Files

A header file is a file that ends in .h (could also end in .hpp but only .h is used in the template) that contains the declarations of functions, classes, and variables without their full implementations. Think of it like a table of contents, it tells other files what exists, while .cpp file is the actual content. To create a header file, follow these steps:

1. Go to the include folder in the template
2. Right click on the include folder
3. Name the file your_name.h (you_name is up to your choice, but it should be the same as the .cpp version of the file.)

After following these steps, you will create a header file. Header files should start with:

```
#pragma once
```

Since everytime you use #include, the compiler duplicates the file. This can result in redefinition of function, multiple definition of variable, and duplicate class definition errors. To prevent this, #pragma once prevents the compiler from repeatedly duplicating the files. Some people might say "I didn't include this and my file still works," and that could be true since these errors mostly happen in larger projects. However, while including guards isn't always required, it is still a good habit to have.

After adding that line, you have to initialize your functions. You only need to initialize the functions you are going to call in other files. For example, in the template, the movement.h header file only has the movement functions. This is because the other functions are only used inside movement.cpp, and as such, aren't required to be in movement.h. To initialize a function in a header file, you follow this format:

```
return_type function_name(argument);
```

Your return_type, function_name, and arguments list has to be **identical** to the function in the .cpp file. One easy way to do this is to copy the function declaration line in the .cpp file, then removing the { bracket, then adding a semicolon (;) at the end.

After you do this, you're done! The header file is this simple to create and write.

File Integration Through #include

To call functions from other files, you first create a header file. After you do so, you can then include the header file in any other file that you will call those functions. The #include syntax is:

```
#include "header_file_name"
```

The include line does not need a semicolon at the end. Some examples from the template include:

```
#include "movement.h"
#include "vex.h"
```

Creating Functions and Calling Functions

Functions are pieces of code that do something. To create a function, follow this format:

```
return_type function_name(argument) {
	//something
}
```

The return_type is the type of data returned when the function ends. For example, in the following function:

```
int getSum(int a, int b) {
	int c = a + b;
	return(c);
}
```

It returns an integer value of the sum of the arguments. Most functions in VEX are void, meaning they do not return anything.

To call a function, follow this format:

```
function_name(argument);
```

And if the function returns a value, you can assign that value to a variable. Using the getSum function example above:

```
int sum = getSum(6, 7);
```

So sum is the value of c.

A function can also call other functions within it. For example:

```
int getSumOfFourNumbers(int a, int b, int c, int d) {
	int e = getSum(a, b);
	int f = getSum(c, d);
	int g = getSum(e, f);
	return(g);
}
```

In this example, the getSum function is called within the getSumOfFourNumbers function. If a function is in the same .cpp file as the function it calls, it does not need a header file.

You can also assign default values to a function. For example:

```
int getSum(int a = 1, int b = 1) {
	int c = a + b;
	return(c);
}
```

In this example if the following is called:

```
int sum = getSum();
```

sum will be 2 because no arguments are passed, so a and b both get initialized to default values of 1. 1 + 1 = 2, then 2 is returned and assigned to the variable sum.

Namespaces

A namespace is basically the library where the function comes from. If you are reading this, you are probably still a beginner, so I wouldn't confuse you with anything about libraries. Basically, every function you use in a VEX program would either be a VEX function or a function you created yourself. A full function call should look like this:

```
namespace::function_name(arguments);
```

But since we are only using VEX and custom functions, you can write this line in your code.

```
using namespace vex;
```

This shortens our function calls to only:

```
function_name(arguments);
```

Since namespace now always defaults to vex unless otherwise specified. An example would be:

```
vex::wait(1, sec);
```

becoming:

```
wait(1, sec);
```

This makes your program more concise.

VEX Concepts

The VEX concepts section will teach you about VEX / C++ objects, how port numbers work, gear ratio's software effect, the difference between 6 watt and 12 watt motors, and how threads work.

Object Oriented Programming and VEX Objects

An object is an instance of a class. A class is like a blueprint of something, while an object is the actual thing built from that blueprint.

Object Oriented Programming

Object oriented programming is the idea that a program is split into layers of abstraction, so different functions don't need to handle the contents of other functions. The benefit of this is so that the code looks cleaner and is easier to debug. OOP is typically centered around the idea of classes and objects. A class is a blueprint while the object is an instance of an object. Classes and objects aren't NECESSARY to write a VEX program, but it is nice to know.

To create a class:

```
class class_name {
	//variables

	//class functions
};
```

For example, if I wanted to create a robot class:

```
class Robot {
	int speed;

	void move() {
		// code to move the robot
	}
};
```

Classes can have a constructor class. A constructor runs when an object is created. The basic syntax is:

```
class_name() {
	//something
}
```

Using the robot example:

```
class Robot {
	int speed;

	//The constructor
	Robot() {
		speed = 0;
	}

	void move() {
		// code to move the robot
	}
};
```

To create an object:

```
class_name object_name;
```

Using the robot example:

```
Robot myRobot;
```

To then use the variables and functions inside the object:

```
object_name.variable_name = value;
object_name.function_name(arguments);
```

Using the robot example:

```
myRobot.speed = 10;
myRobot.move();
```

Normally, classes will also have public and private tags as well as static variables. However, those aren't necessary if you do create classes in a VEX program.

VEX Objects

VEX objects are the motors, the sensors, the pistons, and others. Everything you initialize in robotConfig.cpp is an object. Here are a few examples:

```
brain Brain;
controller Controller1 = controller(primary);
motor Motor = motor(PORT1, ratio6_1, true);
inertial Inertial = inertial(PORT6);
```

```
digital_out Piston = digital_out(Brain.ThreeWirePort.C);
motor_group Motor_group(Motor);
```

Everything initialized here is a VEX object. Hopefully this explains why functions are called like Inertial.heading() instead of heading() in the program.

How Ports Work

Ports are connected by cables to parts. If a part isn't connected to the brain using a cable, then it is not connected to the software. There are mainly two types of ports, the ports on the top of the Brain used to connect motors, sensors, and radio and the three wire ports on the side, which are used to connect piston controllers and other types of sensors.

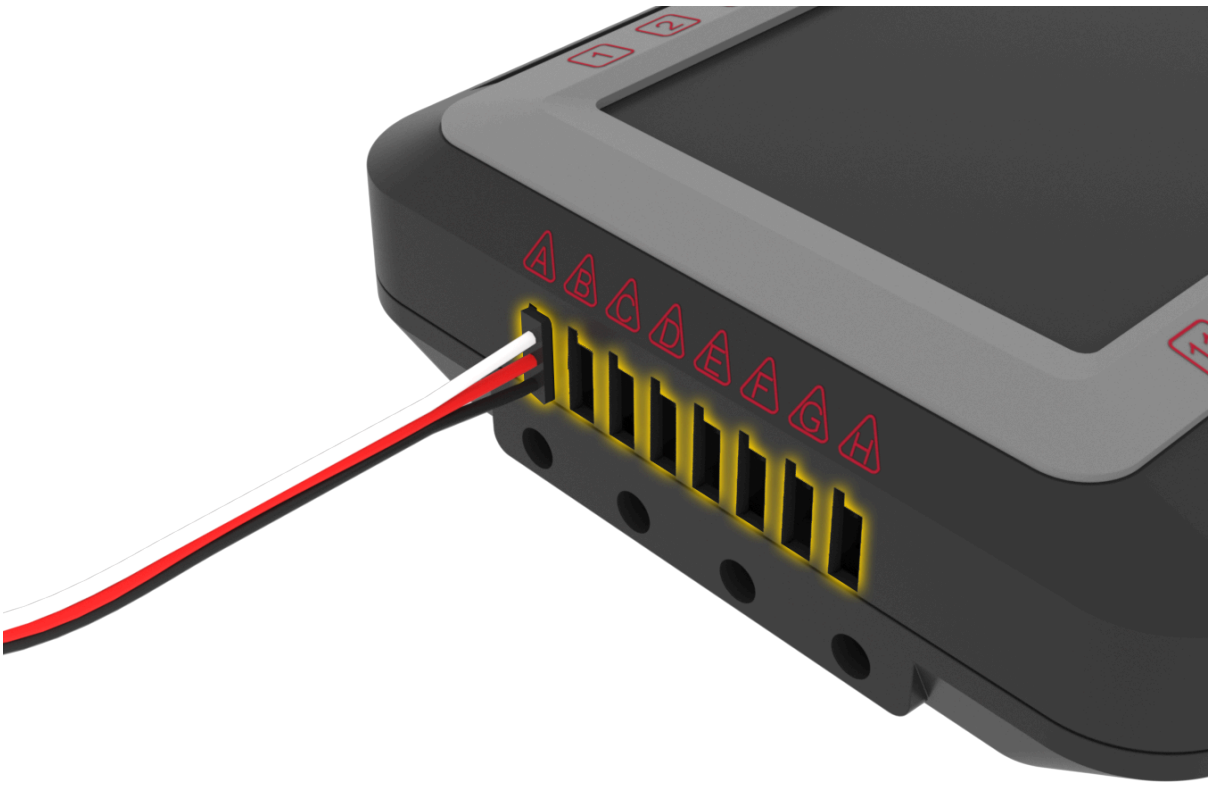
*Ports on the top

For the ports on the top, they are only called through this way:

```
PORT#
```

is the number of the port, for example:

```
PORT6
PORT20
```



*Three wire ports

For the three wire ports, they are actually functions of the brain class, so they are called by:

```
Brain.ThreeWirePort.port_letter
```

The port_letter is the letter of the port, for example:

```
Brain.ThreeWirePort.C
Brain.ThreeWirePort.D
```

Gear Ratio’s Effect on Software

Gear ratio is only present in the move and moveRPM functions of the template and the robot configuration. This is used to calculate the accurate distance. It is best that you confirm with your builder on what gear ratios are used. For more information on template setup, visit the setup section of the guide.

6 vs 12 Watt Motors

The only difference in terms of software between these two types of motors are the gear ratios. The 6 watt motors do not have gear ratios in its initialization arguments while the 12 watt motors do. Both motors can deliver 12 volts though, so there is no difference when using them in terms of software.

VEX Threads

A thread lets your brain / robot execute multiple tasks at the same time. Instead of doing things sequentially, the robot can drive forward, spin an intake, and update the screen all at once. Normally, C++ runs code from top to bottom, but with threads, they can run in parallel. Using the display.h functions, you can see that threads for those are written in main.cpp:

```
thread screenThread(BrainRender);
thread screenThread2(ControllerRender);
```

The general syntax is:

```
thread thread_name(function_name);
```

Apart from the screen render example already in the template, threads aren't frequently used.

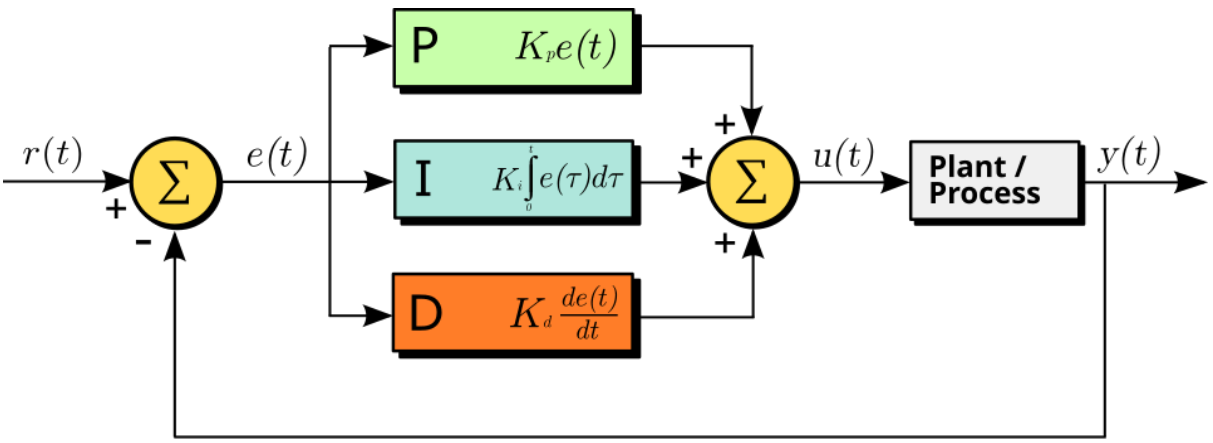
Proportional–Integral–Derivative Controller

Proportional-integral-derivative is commonly shortened to PID. PID is a feedback-based control loop commonly used to manage movement that requires continuous control and automatic adjustment.

Proportional is the P term of PID. It mainly controls the acceleration of the robot. The higher the P, the faster the acceleration.

Integral is the I term of PID. It corrects for accumulated error over time, helping the robot reach its target when P and D alone leave it consistently short of the target.

Derivative is the D term of PID. It mainly controls the deceleration of the robot. The higher the D, the slower the movement.



*A graphic of PID on the Wikipedia page for PID

While I won't go into much detail on how PID works, the main idea of PID is to use the control terms of P, I, and D to influence the output.

$$u(t) = K_{\mathrm{p}} e(t) + K_{\mathrm{i}} \int_0^t e(\tau)\,\mathrm{d}\tau + K_{\mathrm{d}} \frac{\mathrm{d}e(t)}{\mathrm{d}t},$$

*The full control function equation on the Wikipedia page for PID

This is the full control function of PID. While it seems very complex, in terms of code, it really is only this line in the move function of movement.cpp:

```
volt_now = (Kp * distError + Ki * distErrorInt + Kd * (distError - distErrorPrev));
```

The distError is the distance between current position and target. The distErrorInt is the integral term you see in the I section of the PID equation. The (distError - distErrorPrev) is just the derivative term you see in the D section of the PID equation. The sum of these three terms is the final output.

For PID to work well, the P and D has to be balanced well. This requires continuous manual tuning. To tune PID:

1. First set a distance you want to travel such as one meter
2. Set I to 0 and D to 0
3. Increase the P term until the robot slightly overshoots one meter
4. Increase the D term until the robot accurately travels one meter or is short a bit
5. If the robot stops short a bit of one meter but P and D is already very balanced and confirmed through continuous tuning, increase I until the robot accurately travels one meter.

I don't recommend allowing the robot to travel at the max speed of 12 volts as it leads to inaccuracies. In my last VEX competition, I ended up using 6 volts for chassis movement because it was fast enough but still very accurate. The power limit is up to your choice.

This section is only a brief overview on how to use PID. On how PID works, I recommend checking the Wikipedia page (https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller), which is where I learned how to use and write PID.

Integral Windup

The integral term accumulates error over time. This is the integral term from the equation.

$$K_{\mathrm{i}} \int_0^t e(\tau)\,\mathrm{d}\tau$$

*Ki term from the Wikipedia page for PID

If the system has sustained error (which it does in VEX Robotics movement), because the actuator (the motors) is saturated (at 100% already), the integral term keeps accumulating even though the output couldn't physically respond anymore. When the error finally reverses (an overshoot), the integral has built up such a huge value that it must "unwind" (reduce back to 0 from a huge value) before the controller can react in the other direction (correct the overshoot). This causes overshoots, sluggish recovery, and oscillation.

The fundamental idea is that the integral should only accumulate error that the system can handle. When the actuator (the motor) is saturated already (at 100% already), additional accumulation is meaningless and harmful. A cap enforces its usefulness by bounding it to a value that it cannot exceed. The following is the integral windup prevention from the move function in movement.cpp.

```
if (fabs(volt_now) < volt_max) {
        distErrorInt += distError;
}
if (distError * distErrorPrev < 0) {distErrorInt = 0;}
```

What this does is it checks if the motor is already supplying maximum output. If it is, since the motor is now saturated, we stop the integral accumulation. Additionally, the distError * distErrorPrev becomes negative when an overshoot occurs. When an overshoot occurs, we want to reset the integral term to prevent accumulated error from one half of the motion from dragging into the other.

Volt Control vs RPM Control

In general, volt control is much more superior than RPM control because voltage control directly sets how much power the motor gets. More voltage → more speed while less voltage → less speed. This matches how the DC motors of VEX work, which is:

voltage + current → speed + torque

Additionally, with RPM control, the motor tries to stay at a fixed speed and may limit itself to maintain that speed. With voltage control, there is no speed cap from software and motors can spin above rated RPM when unloaded, which results in faster acceleration and higher top-end performance.

Finally, RPM control already uses a built-in PID loop inside the motor. So if you use the moveRPM function, you get a PID inside a PID, which can cause instability and slower response. With voltage control, you control everything yourself. This also leads to more consistency, which is key when tuning PID.

Absolute Heading & Changing Inertial Reading

Absolute heading is the robot's current direction measured by the inertial sensor, which is usually in degrees from 0~360. So absolute heading just represents where the robot is

facing now relative to when the program started. Absolute heading tells you the current direction, while rotation can go past 360 degrees and is tracking the total amount turned. Absolute heading is much easier for writing VEX autonomous programs because you do not have to consider where the robot will end up after some movement like you have to when you use relative heading.

Inertial.setHeading();

Since all the movement functions use an absolute heading system, when your robot starts at an angle that isn't 0 degrees, you will want to use Inertial.setHeading(your_heading); to set the current heading to your liking. Add this to the start of the autonomous function to use.